Программное обеспечение центров обработки данных

Практическая работа №3

Работа с образами в Docker

Один из способов использования Docker процессе разработки и эксплуатации ПО – создание собственных образов и запуск контейнеров на их основе. В предыдущей практической работе мы рассмотрели создание образов при помощи команды commit, однако предпочитаемым способом является создание образов при помощи **Dockerfile**.

Каждый образ Docker – это набор слоев, каждый последующий слой описывает изменения в образе по отношению к предыдущему образу. Все эти изменения можно описать в файле специальными командами и далее образ собирается по инструкциям из файла. Запуск образа в контейнере создает еще один слой, отличающийся от предыдущих тем, что он доступен для записи.

Использование этого подхода позволяет автоматизировать процесс создания новых образов, так как в этом случае все действия, которые необходимо выполнить, чтобы создать образ, записываются в файле-скрипте, который потом можно собирать одной командой.

В данной работе предлагается создать собственный образ Docker при помощи Dockerfile и изучить возможности работы с ним.

Подготовка к практической работе

1. Запустите созданную в рамках Практической работы №1 виртуальную машину с Ubuntu и установленным Docker.
2. После загрузки Ubuntu войдите в систему и запустите терминал (**Ctrl+Alt+T** или поиском по приложениям – **Terminal**).

Создание собственного образа при помощи Dockerfile

3. Для создания dockerfile создайте новую директорию, для чего выполните:

```
mkdir AlpineApache
```

где «AlpineApache» – это название новой директории.

4. Далее перейдите в эту директорию и создайте в ней два новых текстовых файла:

```
cd AlpineApache
> Dockerfile
> index.html
```

где Dockerfile – название файла, в котором будут записаны инструкции для сборки образа. Стандартно выбирается название Dockerfile, однако название можно выбрать и произвольно, но в этом случае при сборке образа необходимо будет указать его.

index.html – файл для тестирования работы web-сервиса.

5. Откройте файл index.html найдя его в проводнике в графическом интерфейсе Ubuntu или через один из редакторов командной строки, например:

```
nano index.html
```

Введите в файле следующий текст и сохраните файл (в nano – Write Out):

```
Hello Docker Apache
```

6. Откройте файл Dockerfile найдя его в проводнике в графическом интерфейсе Ubuntu или через один из редакторов командной строки, например:

```
nano Dockerfile
```

7. Запишите в текстовом файле инструкции, приведенные ниже. Каждая инструкция сопровождается комментарием, который не обязательно копировать в ваш файл, но крайне желательно прочитать и осознать. Также комментарии к некоторым командам приведены после текста файла.

```
# использовать образ alpine как базовый
FROM alpine
```

```
# установить apache2
RUN apk update && apk add \
apache2 \
apache2-ctl

# указать порт, через который к сервисам можно обращаться внутри
# контейнера
EXPOSE 80

# скопировать файл index.html из хостовой директории, где будет
# собираться образ в директорию внутри образа, из которой его
# возьмет веб-сервер Apache
ADD ./index.html /var/www/localhost/htdocs/

# запустить apache2 указав, что он должен работать в приоритетном
# режиме (foreground), а не в фоновом (background)
CMD ["/usr/sbin/apachectl","-D", "FOREGROUND"]
```

Комментарий к команде RUN:

Запуск двух команд одновременно (соединив их оператором И – &&) позволяет при их выполнении создать только один слой образа чтобы не плодить лишние слои без необходимости. Однако те команды, которые могут часто меняться, имеет смысл выносить в отдельный слой, то есть выполнять в отдельной команде RUN. В рамках RUN выполняется или одно действие, или несколько, но соединенных оператором &&

Комментарий к команде EXPOSE:

Команда EXPOSE открывает порт внутри контейнера. Чтобы иметь доступ к этому порту извне, при запуске контейнера необходимо поставить ему в соответствие (mapping) какой-либо внешний номер порта.

Комментарий к команде CMD:

Команда CMD описывает процесс, который будет оставаться активным в контейнере. Контейнер, созданный из образа, будет оставаться запущенным, пока в нем есть активный процесс, поэтому важно запускать процессы в foreground, а не в background.

8. Сохраните текстовый файл. Убедитесь, что в терминале вы находитесь в созданной папке, а затем выполните команду сборки образа:

```
docker build -t alpineapache:1.0 .
```

где alpineapache – имя собираемого образа, а через двоеточие от него указывается версия образа. Если ваш докер файл имеет имя, отличное от Dockerfile, то необходимо указать его в конце вместо «.»

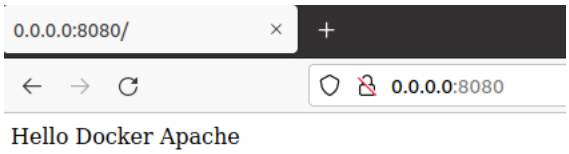
9. После сборки образа запустите его в новом контейнере:

```
docker run -d -p 8080:80 alpineapache:1.0
```

Ключ -d означает, что контейнер будет запущен в фоновом режиме, без получения доступа к нему из терминала.

Ключ -p позволяет опубликовать порты контейнера вовне, при этом далее пишется какой внешний порт (в данном случае 8080), будет соответствовать какому внутреннему порту (80).

10. Для проверки корректности запуска контейнера, откройте браузер и наберите в адресной строке 0.0.0.0:8080. По этому адресу должна открыться страница с содержимым index.html:



11. После завершения работы выключите виртуальную машину закрыв окно и выбрав Suspend или Power off.

Более подробную информацию по шагам в данной Практической работе можно найти по ссылкам:

https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

https://docs.docker.com/engine/reference/builder/